

Laporan Praktikum Minggu Ke-4 Kontrol Cerdas

Minggu ke-4

Nama : Nurzanah
NIM : 224308065
Kelas : TKA 7C
Akun Github (Tautan) : <https://github.com/nurzanahh>

1. Judul Percobaan

Reinforcement Learning for Autonomous Control

2. Tujuan Percobaan

Tujuan dari percobaan pada praktikum kali ini, sebagai berikut :

1. Memahami konsep dasar *Reinforcement Learning* (RL) dalam sistem kendali.
2. Mengimplementasikan agen RL menggunakan algoritma *Deep Q-Network* (DQN).
3. Menggunakan *OpenAI Gym* sebagai simulasi lingkungan untuk pelatihan RL.
4. Melatih dan menguji agen RL untuk mengontrol lingkungan secara otonom.
5. Menggunakan GitHub untuk *version control* dan dokumentasi praktikum.

3. Landasan Teori

a. Reinforcement Learning (RL)

Reinforcement Learning (RL) adalah bagian dari artificial intelligence yang melatih algoritma dengan sistem trial and error. Reinforcement Learning (RL) mengaplikasikan pembelajaran trial and error untuk mencapai target yang diharapkan (Andreanus & Kurniawan, 2018). Selain itu, Reinforcement Learning (RL) juga merupakan sebuah agen (agent) belajar untuk mengambil keputusan dalam suatu lingkungan (environment) dengan cara mencoba (trial and error) dan menerima umpan balik berupa reward (hadiah) atau punishment (hukuman).

b. Deep Q-Network (DQN)

Deep Q-Network (DQN) adalah algoritma Reinforcement Learning (RL) yang menggabungkan algoritma Q-Learning dengan Deep Neural Network (DNN). Pada DQN, DNN digunakan untuk memodelkan Q-value (nilai fungsi aksi Q) dengan menggunakan arsitektur Neural Network (Putra dkk., 2024). Dalam implementasi agen Reinforcement Learning (RL) dengan algoritma Deep Q-Network (DQN) yaitu membangun agen yang belajar mengambil keputusan optimal melalui interaksi dengan lingkungan. Jaringan saraf digunakan untuk memperkirakan nilai aksi (Q-value) pada setiap keadaan, lalu agen memilih aksi dengan strategi. Pengalaman yang diperoleh disimpan dalam replay buffer untuk melatih jaringan secara acak, sementara target network digunakan agar pembaruan lebih stabil.

c. OpenAI Gym Sebagai Simulasi Lingkungan

OpenAI Gym adalah sebuah *toolkit* open source yang dikembangkan oleh OpenAI untuk membangun, menguji, dan membandingkan algoritma Reinforcement Learning (RL). Gym menyediakan berbagai macam environment (lingkungan simulasi) yang bisa digunakan agen RL untuk belajar, mulai dari lingkungan sederhana seperti *CartPole* atau *MountainCar*, hingga yang lebih kompleks seperti *Atari games* dan *robotics simulations*.

OpenAI Gym dalam simulasi lingkungan berperan sebagai platform standar yang menyediakan berbagai environment untuk melatih dan menguji algoritma Reinforcement Learning (RL). Dalam Gym, sebuah lingkungan berfungsi sebagai dunia virtual tempat agen berinteraksi, agen mengirimkan aksi (*action*), lalu lingkungan merespons dengan memberikan keadaan baru (*state*), umpan balik berupa *reward*, serta informasi apakah proses sudah selesai (*done*).

d. Pelatihan agen Reinforcement Learning (RL)

Pada tahap pelatihan, agen berinteraksi secara berulang dengan lingkungan untuk mempelajari strategi atau kebijakan (*policy*) yang optimal. Agen melakukan aksi berdasarkan keadaan tertentu, lalu menerima umpan balik berupa *reward* dan *state* baru. Informasi ini digunakan untuk

memperbarui model internalnya, misalnya melalui pembaruan nilai Q atau parameter jaringan saraf dalam algoritma Deep Q-Network (DQN). Proses ini dilakukan berulang kali hingga agen mampu meningkatkan performanya dan mencapai tujuan tertentu, seperti meminimalkan error atau memaksimalkan total reward.

e. Pengujian agen Reinforcement Learning (RL)

Tahap pengujian dalam Reinforcement Learning dilakukan setelah agen menyelesaikan proses pelatihan dengan tujuan mengevaluasi sejauh mana strategi yang telah dipelajari dapat dijalankan secara konsisten. Pada tahap ini, agen tidak lagi difokuskan untuk mencari atau mencoba aksi baru, melainkan menggunakan pengetahuan yang sudah diperoleh untuk menyelesaikan tugas seefektif mungkin. Evaluasi biasanya dilakukan dengan menjalankan agen pada lingkungan yang sama atau lingkungan baru yang mirip, kemudian mengukur kinerjanya berdasarkan rata-rata reward yang diperoleh, tingkat keberhasilan dalam mencapai tujuan, serta stabilitas performa dari satu episode ke episode lainnya.

4. Analisis dan Diskusi

Analisis

a. Performa agen dalam mengontrol environment CartPole

Dalam pengujian pada *CartPole environment*, agen RL menunjukkan kemampuan adaptasi yang baik, di mana performanya meningkat seiring jumlah episode pelatihan hingga mampu menjaga tiang tetap seimbang dalam waktu yang lama. Efektivitas kinerja agen sangat dipengaruhi oleh parameter pelatihan.

b. Pengaruh parameter

- Nilai *gamma* : menentukan orientasi terhadap reward jangka panjang
- *Learning rate* : memengaruhi stabilitas konvergensi, dan pengaturan eksplorasi berperan dalam pencarian strategi optimal.
- *Epsilon* : mengatur keseimbangan antara eksplorasi dan eksploitasi.

c. Tantangan dalam pelatihan agen RL

Proses pelatihan menghadapi beberapa tantangan, antara lain kestabilan pembelajaran, keseimbangan eksplorasi–eksploitasi, serta tingginya kebutuhan sumber daya komputasi.

Diskusi

a. Perbedaan RL dan metode supervised learning dalam sistem kendali

Perbedaan utama antara Reinforcement Learning (RL) dan supervised learning dalam sistem kendali adalah pada cara agen belajar. Supervised learning membutuhkan data berlabel dengan pola input-output yang jelas, sedangkan RL tidak memiliki label dan agen belajar melalui interaksi langsung dengan lingkungan melalui mekanisme trial-and-error. Umpan balik berupa reward digunakan untuk menyesuaikan kebijakan hingga tujuan tercapai.

b. Optimalisasi Strategi eksplorasi (exploration) dan eksploitasi (exploitation) agen RL

Strategi eksplorasi dan eksploitasi penting agar agen tidak hanya mengulang strategi lama, tetapi juga mencoba hal baru. Optimalisasi dilakukan dengan metode seperti *epsilon decay*, *upper confidence bound (UCB)*, atau *Boltzmann exploration*. Dengan cara ini, agen lebih banyak mengeksplorasi di awal pelatihan lalu beralih ke eksploitasi saat sudah menemukan strategi yang efektif.

c. Potensi aplikasi lain dari RL dalam sistem kendali nyata

Aplikasi RL dalam sistem kendali nyata sangat luas, misalnya pengendalian robot otonom, manajemen energi pada *smart grid*, optimasi transportasi seperti kereta dan kendaraan otonom, hingga kendali proses industri yang dinamis.

5. Assignment

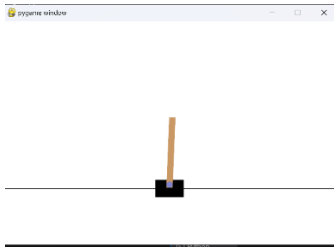
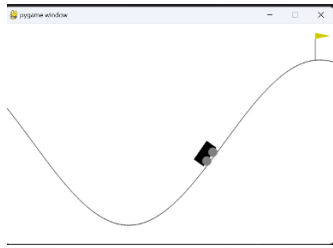
Pada tahap assignment di praktikum 4 ini, algoritma Deep Q-Network (DQN) yang sebelumnya digunakan dimodifikasi dengan menambahkan target network guna meningkatkan stabilitas proses pelatihan. Kehadiran target network membuat pembaruan nilai Q lebih teratur, sehingga agen dapat belajar secara lebih konsisten tanpa menimbulkan osilasi besar

selama training. Selain itu, pengujian agen tidak terbatas pada environment CartPole-v1, melainkan juga dilakukan pada environment lain seperti LunarLander-v2 dan MountainCar-v0. Hasil eksperimen menunjukkan bahwa setiap environment memiliki tingkat kompleksitas yang berbeda. Pada CartPole, agen relatif cepat memperoleh strategi untuk menjaga keseimbangan, sedangkan pada LunarLander dibutuhkan jumlah episode yang lebih banyak agar agen mampu menghasilkan manuver pendaratan yang optimal. Adapun MountainCar memberikan tantangan tersendiri karena agen harus memanfaatkan momentum untuk mencapai puncak bukit, sehingga proses belajar akan lebih lambat.

6. Data dan Output Hasil Pengamatan

Data dan hasil yang diperoleh dalam pengamatan :

No	Variabel	CartPole-v1	MountainCar-v0
1	Waktu Pelatihan	Agen cepat mencapai performa optimal setelah beberapa ratus episode	Agen membutuhkan jumlah episode yang lebih banyak karena kompleksitas lingkungan lebih tinggi dan reward sulit diperoleh.
2	Jumlah Episode	CartPole ditraining hingga 1000 episode <pre> _name_ == "__main__": env = gym.make("CartPole-v1", render_mode=None) state_size = env.observation_space.shape[0] action_size = env.action_space.n agent = DQNAgent(state_size, action_size) episodes = 1000 batch_size = 32 </pre>	MountainCar di training hingga 1000 episode <pre> _name_ == "__main__": # pakai _name_ env = gym.make("MountainCar-v0", render_mode="human") state_size = env.observation_space.shape[0] action_size = env.action_space.n agent = DQNAgent(state_size, action_size) episodes = 1000 batch_size = 32 </pre>
3	Skor per Episode (Testing)	Skor rata-rata meningkat signifikan (contoh : 8, 9, 10, 11 hingga >200) <pre> Episode 2: Score = 9 Episode 3: Score = 10 Episode 4: Score = 9 Episode 5: Score = 10 Episode 6: Score = 10 Episode 7: Score = 10 Episode 8: Score = 8 Episode 9: Score = 9 Episode 10: Score = 11 </pre>	Skor awal mencapai 199 karena maksimal langkah dalam satu episode adalah 200 <pre> Episode: 1/1000, Score: 199, Epsilon: 1.00 Episode: 2/1000, Score: 199, Epsilon: 0.99 Episode: 3/1000, Score: 199, Epsilon: 0.99 Episode: 4/1000, Score: 199, Epsilon: 0.99 Episode: 14/1000, Score: 199, Epsilon: 0.94 Episode: 15/1000, Score: 199, Epsilon: 0.93 Episode: 16/1000, Score: 199, Epsilon: 0.93 Episode: 17/1000, Score: 199, Epsilon: 0.92 Episode: 18/1000, Score: 199, Epsilon: 0.92 Episode: 19/1000, Score: 199, Epsilon: 0.91 Episode: 20/1000, Score: 199, Epsilon: 0.91 </pre>

4	Kinerja Agen	Agen semakin stabil dengan skor yang makin tinggi	Agen masih menghadapi tantangan untuk mencapai puncak bukit
5	Kondisi Berhasil	Episode berakhir jika tiang jatuh atau batas langkah maksimum tercapai	Episode berakhir jika mobil berhasil sampai puncak bukit (flag)
6	Output		

7. Kesimpulan

Hasil pengujian menunjukkan bahwa algoritma Deep Q-Network (DQN) mampu digunakan secara efektif untuk mengendalikan lingkungan simulasi dengan karakteristik yang berbeda. Pada *CartPole-v1*, agen relatif cepat mencapai kinerja optimal karena pola reward yang konsisten mampu mempermudah proses pembelajaran. Sebaliknya, pada *MountainCar-v0*, agen membutuhkan jumlah episode yang lebih banyak serta strategi eksplorasi yang lebih baik untuk mencapai tujuan. Hal ini menunjukkan bahwa semakin kompleks suatu environment, semakin besar pula tantangan yang harus dihadapi oleh agen dalam pembelajaran Reinforcement Learning (RL).

8. Saran

1. Untuk menjaga konsistensi selama proses pelatihan, penerapan teknik seperti target network serta experience replay dengan kapasitas penyimpanan yang lebih luas direkomendasikan, sehingga pembaruan nilai Q menjadi lebih stabil dan tidak terlalu dipengaruhi oleh hubungan antar data jangka pendek.
2. Selain itu, penting untuk memperluas kajian dengan membandingkan algoritma RL alternatif, seperti Policy Gradient, Actor-Critic, maupun Proximal Policy Optimization (PPO). Perbandingan tersebut akan memberikan gambaran lebih jelas mengenai keunggulan dan keterbatasan

relatif tiap algoritma, terutama pada lingkungan yang memiliki tingkat kompleksitas lebih tinggi.

9. Daftar Pustaka

- Andreanus, J., & Kurniawan, A. (2018). Sejarah, Teori Dasar dan Penerapan Reinforcement Learning: Sebuah Tinjauan Pustaka. *Jurnal Telematika*, 12(2), 113–118. <https://doi.org/10.61769/telematika.v12i2.193>
- Putra, R. A., Syahbana, Y. A., & Ananda. (2024). Implementasi Algoritma Deep Q-Network (DQN) pada Lampu Lalu Lintas Adaptif Berdasarkan Waktu Tunggu dan Arus Kendaraan. *The Indonesian Journal of Computer Science*, 13(5). <https://doi.org/10.33022/ijcs.v13i5.4372>